

SYNTH-01: Synthetic Monitoring Fundamentals

Series: SYNTH — Synthetic Monitoring | Notebook: 1 of 6 | Created: December 2025 | Last Updated: 07/30/2026

Understanding Proactive Availability and Performance Testing

This notebook introduces Dynatrace Synthetic Monitoring, which enables proactive testing of application availability, functionality, and performance from locations around the world.

Table of Contents

- [1. What is Synthetic Monitoring?](#)
 - [2. Monitor Types](#)
 - [3. Key Concepts](#)
 - [4. Synthetic Data in Grail](#)
 - [5. Your First Synthetic Query](#)
-

Prerequisites

- ✓ Access to a Dynatrace environment with Synthetic Monitoring enabled
- ✓ DQL query permissions (viewer role minimum)
- ✓ Basic understanding of web applications and APIs

Sprint 1.337 (April 2026) Updates

Two sprint-1.337 changes touch synthetic monitoring:

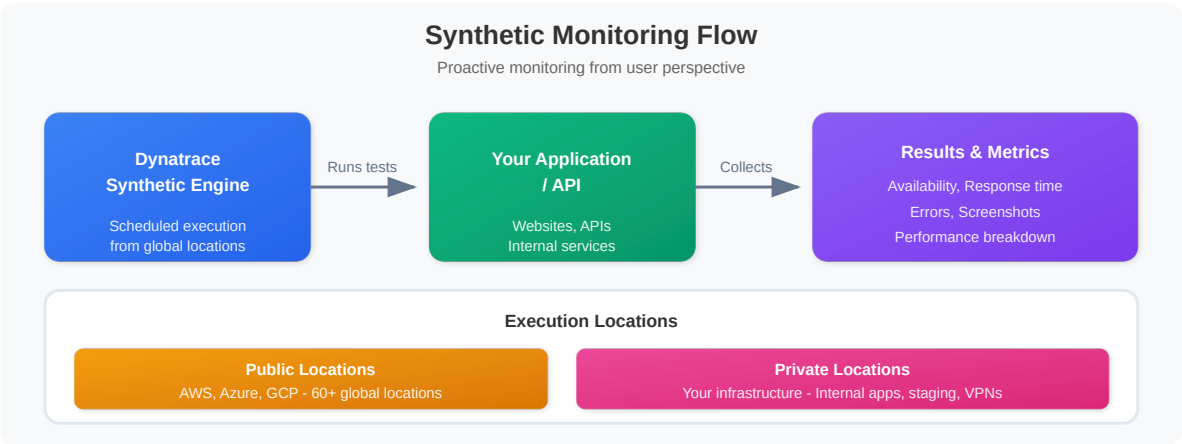
- **OneAgent primary fields/tags at the source** (Latest Dynatrace) — when synthetic monitors are paired with private-location ActiveGates running on monitored hosts (SYNTH-04), the host's primary tags ride on synthetic execution metadata. Useful for cost-attribution dashboards that want to bill synthetic execution by team or product line.
 - **Configuration API → Settings v2 + Platform tokens** for new automation. Synthetic monitor management via Terraform/Monaco should target Settings v2 (`builtin:synthetic_test`) — note the v1.88.0 Terraform provider boundary still requires API tokens for synthetic monitors specifically (see AUTOM-04 § Authentication Boundary).
-

1. What is Synthetic Monitoring?

Synthetic Monitoring simulates user interactions with your applications from external locations, providing:

- **Proactive Detection:** Find issues before real users encounter them
- **24/7 Availability Testing:** Monitor even when no users are active
- **Global Performance Baseline:** Measure response times from multiple locations

- **SLA Validation:** Verify service level agreements are being met
- **Third-Party Dependency Monitoring:** Test external APIs and services



Synthetic vs Real User Monitoring (RUM)

Aspect	Synthetic Monitoring	Real User Monitoring
Data Source	Simulated transactions	Actual user sessions
Coverage	24/7, consistent	Only when users active
Locations	Controlled, specific	Wherever users are
Use Case	Baseline, SLA, proactive	Actual experience
Cost	Per execution	Per session

2. Monitor Types

Dynatrace offers three types of synthetic monitors:

Browser Monitors (Single-URL and Browser Clickpath)

Type	Description	Use Case
Single-URL	Load a single page	Homepage availability
Browser Clickpath	Multi-step user journey	Login flows, checkout

Capabilities:

- Full browser rendering (Chrome)
- JavaScript execution
- Visual validation (screenshots)
- Performance metrics (W3C timing)
- Resource waterfall analysis

HTTP Monitors

Type	Description	Use Case
Single Request	One HTTP call	API health check

Type	Description	Use Case
Multi-step	Chained requests	API workflow validation

Capabilities:

- Any HTTP method (GET, POST, PUT, DELETE)
- Custom headers and authentication
- Response validation (status, content, JSON)
- SSL certificate monitoring
- Lightweight and fast execution

Third-Party Monitors

Integration with external synthetic providers:

- Catchpoint
- Pingdom
- Site24x7

3. Key Concepts

Execution Locations

Location Type	Description	Best For
Public	Dynatrace-hosted worldwide	External-facing apps
Private	Your infrastructure	Internal apps, security

Execution Frequency

How often the monitor runs:

- **Browser:** 5, 10, 15, 30, 60 minutes
- **HTTP:** 1, 5, 10, 15, 30, 60 minutes

Outage Detection

Dynatrace detects outages based on:

- Consecutive failures from single location
- Failures from multiple locations simultaneously
- Local outage vs global outage classification

Key Metrics

Synthetic measurements are surfaced two ways in Grail (see §4): as **metric keys** queried with `timeseries` , and as fields on execution **events** queried with `fetch dt.synthetic.events` . The table maps each concept to both.

Concept	Metric key (<code>timeseries</code>)	Event field (<code>fetch dt.synthetic.events</code>)	Monitor Type
Availability (%)	<code>dt.synthetic.http.availability / dt.synthetic.browser.availability</code>	derived from <code>result.state == "SUCCESS"</code>	All
Total duration	<code>dt.synthetic.http.duration / dt.synthetic.browser.duration</code>	<code>result.statistics.duration</code>	All

Concept	Metric key (<code>timeseries</code>)	Event field (<code>fetch dt.synthetic.events</code>)	Monitor Type
DNS lookup	<code>dt.synthetic.http.dns</code>	<code>result.statistics.host_name_resolution_time</code>	HTTP
TCP connect	<code>dt.synthetic.http.tcp_connect_time</code>	<code>result.statistics.tcp_connect_time</code>	HTTP
TLS handshake	<code>dt.synthetic.http.tls_handshake_time</code>	<code>result.statistics.tls_handshake_time</code>	HTTP
Time to first byte	<code>dt.synthetic.http.time_to_first_byte</code>	<code>result.statistics.time_to_first_byte</code>	HTTP
Executions (count)	<code>dt.synthetic.http.executions</code> / <code>dt.synthetic.browser.executions</code>	<code>count()</code> over the events	All

Note on units: the `dt.synthetic.*.duration` *metric* is already expressed in milliseconds. The `result.statistics.*` *event* fields are `duration` -typed (nanoseconds) — divide by `1ms` to get milliseconds, e.g. `result.statistics.duration / 1ms`.

4. Synthetic Data in Grail

With **Synthetic on Grail**, execution results are stored as **synthetic events** in Grail and are also rolled up into **synthetic metrics**. There are two complementary query paths:

Path 1 — Execution events (`fetch dt.synthetic.events`)

One record per execution (and per step). This is where per-failure diagnostics, status messages, SSL certificate data, and per-step timings live.

Data object	Description
<code>dt.synthetic.events</code>	Monitor- and step-level execution results (HTTP, browser, multi-protocol)
<code>dt.synthetic.detailed_events</code>	Detailed per-step records (request-level breakdowns)

Records are distinguished by `event.type` :

<code>event.type</code>	Monitor	Granularity
<code>http_monitor_execution</code>	HTTP	One per execution (carries <code>result.state</code>)
<code>http_step_execution</code>	HTTP	One per request/step
<code>browser_monitor_execution</code>	Browser	One per execution
<code>browser_step_execution</code>	Browser	One per clickpath step
<code>multiprotocol_monitor_execution</code>	Network (ICMP/DNS/TCP)	One per execution

Path 2 — Metrics (`timeseries`)

Pre-aggregated series for dashboards and trends — already scaled to milliseconds / percent.

Metric key	Description
<code>dt.synthetic.http.availability / dt.synthetic.browser.availability</code>	Availability rate (%)
<code>dt.synthetic.http.duration / dt.synthetic.browser.duration</code>	Total duration (ms)
<code>dt.synthetic.http.dns</code> , <code>.tcp_connect_time</code> , <code>.tls_handshake_time</code> , <code>.time_to_first_byte</code>	HTTP timing breakdown (ms)
<code>dt.synthetic.multi_protocol.executions</code>	Network monitor execution count

Key fields on `dt.synthetic.events`

Field	Type	Description
<code>dt.synthetic.monitor.id</code>	string	Monitor ID
<code>monitor.name</code>	string	Monitor display name
<code>step.name</code>	string	Step display name
<code>dt.entity.http_check</code>	string	HTTP monitor entity
<code>dt.entity.synthetic_location</code>	string	Location entity (resolve with <code>entityName()</code>)
<code>result.state</code>	string	SUCCESS / FAIL (on monitor-execution events)
<code>result.status.message</code>	string	e.g. HEALTHY , UNKNOWN_HOST
<code>result.status.code</code>	long	0 = success; non-zero = failure
<code>result.statistics.duration</code>	duration	Total execution time
<code>result.statistics.response_status_code</code>	long	HTTP response code

Path 3 — Entity definitions (classic tables and their Smartscape equivalents)

Entity tables (`fetch dt.entity.synthetic_test` , `fetch dt.entity.http_check` , `fetch dt.entity.synthetic_location`) hold monitor and location *definitions* — use them to enumerate configured monitors and resolve names. They still exist and still return rows.

Each of them now also has a **Smartscape node type**, and that is the preferred surface. SaaS 1.344 (released 07/27/2026, with a **staged tenant rollout from 07/29/2026**) moved the Digital Experience apps — Synthetic among them — so their queries and filters "primarily use `dt.smartscape.*` entities, instead of `dt.entity.*`". Verify 1.344 has reached your tenant before relying on the app-side change; the node types themselves are already queryable via DQL regardless, and cost nothing to read (Smartscape queries scan 0 bytes).

Classic entity	Smartscape node type	Smartscape model
<code>dt.entity.synthetic_test</code> (browser monitors)	BROWSER_MONITOR plus BROWSER_MONITOR_STEP	<code>dt.smartscape.browser_monitor / ..._step</code>
<code>dt.entity.http_check</code>	HTTP_MONITOR plus HTTP_MONITOR_STEP	<code>dt.smartscape.http_monitor / ..._step</code>
<code>dt.entity.multiprotocol_monitor</code>	NETWORK_AVAILABILITY_MONITOR	<code>dt.smartscape.network_availability_monitor</code>
<code>dt.entity.synthetic_location</code>	SYNTHETIC_LOCATION	<code>dt.smartscape.synthetic_location</code>

Two rows in that table cannot be guessed from the classic name, and both fail silently if you guess wrong:

- `multiprotocol_monitor` **was renamed, not transliterated**. The node type is `NETWORK_AVAILABILITY_MONITOR`. `smartscapeNodes "MULTIPROTOCOL_MONITOR"` does **not** raise an error — it returns zero rows, which reads exactly like "this environment has no network monitors".
- `synthetic_test` **splits into two node types**. Clickpath steps are their own nodes (`BROWSER_MONITOR_STEP`), joined to the monitor by a `contains` edge (or `belongs_to` from the step's side) — they are not attributes of the monitor record. Any query that treated `step.name` as a field of the test needs a `traverse` on Smartscape. The same applies to `HTTP_MONITOR_STEP`.

Two smaller conveniences: the node's `name` field is already the display name, so the `entityName()` resolution step that classic entity queries needed disappears; and every node carries `id_classic` (`SYNTHETIC_TEST-*`, `HTTP_CHECK-*`, `SYNTHETIC_LOCATION-*`), which is the join key between migrated and unmigrated queries during the transition.

Migration mechanics — for the general rules on classifying a query before migrating it (dimension-first for mass data, `smartscapeNodes` only for entity-list queries), the type and construct mapping, `traverse`, and the `id_classic` bridge, see **FAQ-16: Migrating Classic Entity Selectors to Smartscape**.

Sources: Dynatrace SaaS release notes 1.344 (DT docs) — "All latest Dynatrace queries and filters in the above apps now primarily use `dt.smartscape.*` entities, instead of `dt.entity.*` and `dt.rum.application.*`". **Derived:** the classic-to-node mapping table is read from `dt.semantic_dictionary.models ( data_object == "smartscape.nodes" , classic_models expanded )` on a live tenant, 07/30/2026; the two "cannot be guessed" traps are the observed consequences of that mapping, not documented warnings.

5. Your First Synthetic Query

Let's explore synthetic monitoring data in your environment.

```
// Discover configured synthetic monitors in your environment
// PREFERRED -- Smartscape. Browser monitors are BROWSER_MONITOR; the node's `name`
// field IS the display name, so no entityName() resolution is needed.
smartscapeNodes "BROWSER_MONITOR"
| fields name, id, id_classic, url, enabled, frequency
| sort name asc
| limit 50

// Other monitor types:
//   HTTP monitors -> smartscapeNodes "HTTP_MONITOR"
//   Network availability monitors -> smartscapeNodes "NETWORK_AVAILABILITY_MONITOR"
// NOT "MULTIPROTOCOL_MONITOR" -- that node type does not exist and returns zero rows
// rather than an error, which is indistinguishable from having no network monitors.

// FALLBACK (classic surface -- still functional):
// fetch dt.entity.synthetic_test
// | fields id, entity.name
// | sort entity.name asc
// | limit 50
```

```
// Count executions by monitor type (from synthetic events)
// event.type distinguishes HTTP / browser / multi-protocol executions
fetch dt.synthetic.events, from: now() - 24h
| summarize count = count(), by: {event.type}
| sort count desc
```

```
// List available synthetic locations
// PREFERRED -- Smartscape. location_type, stage, cloud.provider and geo.* are all
// available on the SYNTHETIC_LOCATION node. The classic entity genuinely has none of
// them, which is why the old guidance said these fields "are not available".
smartscapeNodes "SYNTHETIC_LOCATION"
| fields name, location_type, stage, cloud.provider, geo.country.name, geo.city.name
| sort name asc
| limit 50

// See SYNTH-04 for the private-vs-public breakdown and a verification caveat on the
// exact location_type literals your tenant returns.

// FALLBACK (classic surface -- still functional, but name-only: cloudPlatform, city
// and countryCode really are absent from dt.entity.synthetic_location):
// fetch dt.entity.synthetic_location
// | fields id, entity.name
// | sort entity.name asc
// | limit 50
```

```
// Recent synthetic execution results (last 24 hours)
// Monitor-level events carry result.state; one row per execution
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| fields timestamp,
    monitor = monitor.name,
    type = event.type,
    location = entityName(dt.entity.synthetic_location),
    state = result.state,
    status = result.status.message,
    response_ms = result.statistics.duration / 1ms
| sort timestamp desc
| limit 100
```

```
// Synthetic availability summary (last 24 hours)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    total_executions = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
}
| fieldsAdd availability_pct = round((successful * 100.0) / total_executions, decimals: 2)
```

```
// Response time by monitor (last 24 hours)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter isNotNull(result.statistics.duration)
| summarize {
    avg_response_ms = avg(result.statistics.duration / 1ms),
    max_response_ms = max(result.statistics.duration / 1ms),
    executions = count()
}, by: {monitor.name}
| fieldsAdd avg_response_ms = round(avg_response_ms, decimals: 1)
| fieldsAdd max_response_ms = round(max_response_ms, decimals: 1)
| sort avg_response_ms desc
| limit 20
```

Summary

In this notebook, you learned:

- ✓ **What Synthetic Monitoring is** and how it differs from RUM
 - ✓ **Monitor types** - Browser (single-URL, clickpath), HTTP, and network (multi-protocol) monitors
 - ✓ **Key concepts** - Locations, frequency, outage detection
 - ✓ **Grail data model** - `fetch dt.synthetic.events` for execution detail and `timeseries dt.synthetic.*` for metrics
 - ✓ **Basic DQL queries** - Discover monitors, locations, and results with `result.state` / `result.statistics.*`
-

Next Steps

Continue to **SYNTH-02: Browser Monitors** to learn how to create and optimize browser-based synthetic tests.

References

- [Synthetic on Grail \(DT docs\)](#)
 - [Synthetic app \(DT docs\)](#)
 - [Synthetic on Grail metrics \(DT docs\)](#)
 - [Browser monitors \(DT docs\)](#)
 - [HTTP monitors \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.